ps3p2

February 26, 2026

Consider the model studied in Problem 3 on Problem set 2, where (Y, X) follow the logit model

$$Y_1 = \mathbf{1}\{\beta_{01} + X_1\beta_{02} + V_1 \geq 0\}, \quad (1)$$

where V_1 is an unobservable variable independent of X_1 with the logit c.d.f. $\Lambda(v) = \frac{\exp(v)}{1+\exp(v)}$, and X_1 is a Bernoulli random variable such that $P(X_1 = 1) = 0.75$. The parameter $\beta_0 = (\beta_{01}, \beta_{02})$ denotes the true population values and $\beta = (\beta_1, \beta_2)$ denotes a generic parameter vector, possibly different from β_0. Suppose $\beta_{01} = \beta_{02} = 1$.Write a function named LogitDraws that draws 1000 realizations of V_1 and X_1. For each draw of V_1 and X_1 then compute Y_1 according to (1), and output from the function two 1000×1 vectors y and x containing all the realizations of Y_1 and X_1, respectively.

[1]:
```
import numpy as np

def LogitDraws():
    """
    Draws 1000 realizations of V_1 and X_1, computes Y_1, and returns the draws.

    The model is Y_1 = 1{beta_01 + X_1*beta_02 + V_1 >= 0},
    where V_1 ~ Logistic(0, 1), X_1 ~ Bernoulli(0.75),
    and beta_01 = beta_02 = 1.

    Returns:
        y (np.ndarray): 1000x1 vector of Y_1 realizations.
        x (np.ndarray): 1000x1 vector of X_1 realizations.
    """
    # Number of draws
    n_draws = 1000

    # True parameter values
    beta_01 = 1
    beta_02 = 1

    # Draw 1000 realizations of X_1 from a Bernoulli distribution with p=0.75
    x = np.random.binomial(1, 0.75, n_draws)

    # Draw 1000 realizations of V_1 from a standard logistic distribution
    v = np.random.logistic(loc=0, scale=1, size=n_draws)

    # Compute Y_1
```

```
    y = (beta_01 + x * beta_02 + v >= 0).astype(int)

    # Reshape to be column vectors (1000x1)
    y = y.reshape(-1, 1)
    x = x.reshape(-1, 1)

    return y, x
```

[2]:
```
# Call the function and store the results
y_draws, x_draws = LogitDraws()

# Print the shapes to verify
print("Shape of y_draws:", y_draws.shape)
print("Shape of x_draws:", x_draws.shape)

# Print the first 10 draws as a sample
print("\nSample draws (first 10):")
print("Y | X")
print("-----")
for i in range(10):
    print(f"{y_draws[i][0]} | {x_draws[i][0]}")
```

```
Shape of y_draws: (1000, 1)
Shape of x_draws: (1000, 1)

Sample draws (first 10):
Y | X
-----
1 | 1
1 | 0
1 | 1
1 | 1
0 | 1
1 | 1
1 | 0
1 | 1
1 | 0
1 | 0
```

[3]:
```
import scipy.optimize as opt
from scipy import special # Use special.expit for the logistic function

# Assuming y_draws and x_draws are available from the previous cell

# Add a constant to the exogenous variables for the intercept
X = np.hstack((np.ones((x_draws.shape[0], 1)), x_draws))
y = y_draws.flatten()
```

```python
# Define the negative log-likelihood function for Logit
def neg_log_likelihood_logit(beta, y, X):
    # Calculate the linear combination of predictors
    linear_pred = X @ beta
    # Calculate the probability of Y=1 using the logistic CDF (also called expit␣
↪or sigmoid)
    p_y1 = special.expit(linear_pred)
    # The log-likelihood function
    # To avoid log(0) errors, we use a small epsilon
    epsilon = 1e-9
    log_likelihood = np.sum(y * np.log(p_y1 + epsilon) + (1 - y) * np.log(1 -␣
↪p_y1 + epsilon))
    # Return the negative of the log-likelihood because we are minimizing
    return -log_likelihood

# Initial guess for the parameters (beta_1, beta_2)
initial_beta = np.zeros(X.shape[1])

# Perform the MLE estimation
result = opt.minimize(
    neg_log_likelihood_logit, # Use the correct logit function
    initial_beta,
    args=(y, X),
    method='BFGS'
)

if result.success:
    mle_beta = result.x
    print("MLE estimates for beta (Logit Model):")
    print(f"beta_1 (intercept): {mle_beta[0]:.4f}")
    print(f"beta_2 (coefficient for X_1): {mle_beta[1]:.4f}")
else:
    print("MLE estimation failed.")
    print(result.message)
```

```
MLE estimates for beta (Logit Model):
beta_1 (intercept): 0.8457
beta_2 (coefficient for X_1): 0.8895
```

[4]:
```python
# This cell calculates the asymptotic variance of the MLE estimators using the␣
↪BHHH estimator.
# It depends on the 'mle_beta', 'y', and 'X' variables from the previous cell.

import numpy as np
from scipy import special

# Calculate p_i for each observation using the estimated beta
```

```python
z = X @ mle_beta
p = special.expit(z) # Use logistic CDF

# To avoid division by zero in case p is 0 or 1
epsilon = 1e-9
p = np.clip(p, epsilon, 1 - epsilon)

# Calculate the score (gradient) for each observation for the Logit model
# g is a (n_obs, n_params) matrix
p_reshaped = p.reshape(-1, 1)
y_reshaped = y.reshape(-1, 1)

# The score for the logit model is simply (y - p)X
g = (y_reshaped - p_reshaped) * X

# Calculate the BHHH estimator of the Information Matrix
# J = sum(g_i' * g_i)
J = g.T @ g

# The asymptotic variance-covariance matrix is the inverse of J
try:
    asymptotic_var_cov_bhhh = np.linalg.inv(J)

    # The diagonal elements are the variances of the estimators
    variances_bhhh = np.diag(asymptotic_var_cov_bhhh)

    # The standard errors are the square roots of the variances
    std_errors_bhhh = np.sqrt(variances_bhhh)

    print("Asymptotic Variance-Covariance Matrix (BHHH estimator for Logit):")
    print(asymptotic_var_cov_bhhh)
    print("\n")

    print("Asymptotic Variances of estimators (BHHH for Logit):")
    print(f"Var(beta_1_hat): {variances_bhhh[0]:.4f}")
    print(f"Var(beta_2_hat): {variances_bhhh[1]:.4f}")
    print("\n")

    print("Standard Errors of estimators (BHHH for Logit):")
    print(f"SE(beta_1_hat): {std_errors_bhhh[0]:.4f}")
    print(f"SE(beta_2_hat): {std_errors_bhhh[1]:.4f}")

except np.linalg.LinAlgError:
    print("Could not compute the variance-covariance matrix.")
    print("The Information Matrix (J) may be singular.")
```

```
Asymptotic Variance-Covariance Matrix (BHHH estimator for Logit):
[[ 0.01624169 -0.01624169]
```

```
 [-0.01624169  0.02733954]]


Asymptotic Variances of estimators (BHHH for Logit):
Var(beta_1_hat): 0.0162
Var(beta_2_hat): 0.0273


Standard Errors of estimators (BHHH for Logit):
SE(beta_1_hat): 0.1274
SE(beta_2_hat): 0.1653
```

0.0.1 Formulas for Asymptotic Variance (Logit Model)

To calculate the asymptotic variance of the MLE estimators, we used the **Berndt–Hall–Hall–Hausman (BHHH)** method. Here are the formulas involved:

1. Score Vector (Gradient of the Log-Likelihood) First, for each observation `i`, we compute the score vector `g_i`, which is the gradient of the log-likelihood function for that observation, evaluated at the MLE estimates (`\hat{\beta}`).

The formula for the score `g` in the **Logit** model is remarkably simple:

$$g_i = (y_i - \Lambda(X_i'\hat{\beta}))X_i$$

Where: - `y` and `X` are the data for the i-th observation. - `Λ` (Lambda) is the Cumulative Distribution Function (CDF) of the standard logistic distribution.

2. Information Matrix (BHHH Estimator) Next, we estimate the Information Matrix (`J`) by summing the outer product of each observation's score vector with itself:

$$J = \sum_{i=1}^{n} g_i g_i'$$

This is what the line `J = g.T @ g` in the code calculates.

3. Asymptotic Variance-Covariance Matrix Finally, the asymptotic variance-covariance matrix (`V`) is calculated by taking the inverse of the Information Matrix `J`:

$$V(\hat{\beta}) = J^{-1}$$

The diagonal elements of this resulting matrix `V` are the variances for each of your estimated parameters (`\hat{\beta}` and `\hat{\beta}`). The square root of these diagonal elements gives the standard errors.

[5]:
```
# This cell calculates the 95% confidence interval for beta_2_hat.
# It depends on 'mle_beta' and 'std_errors_bhhh' from the previous cells.
```

```
# The z-score for a 95% confidence interval is approximately 1.96
z_score = 1.96

# Get the MLE estimate and standard error for beta_2 from previous calculations
beta_2_hat = mle_beta[1]
se_beta_2_hat = std_errors_bhhh[1]

# Calculate the lower and upper bounds of the confidence interval
lower_bound = beta_2_hat - z_score * se_beta_2_hat
upper_bound = beta_2_hat + z_score * se_beta_2_hat

print("95% Confidence Interval for beta_2_hat (from Logit model):")
print(f"[{lower_bound:.4f}, {upper_bound:.4f}]")
```

```
95% Confidence Interval for beta_2_hat (from Logit model):
[0.5654, 1.2136]
```

[6]:
```
# This cell runs a Monte Carlo simulation using the CORRECT Logit estimation.
import numpy as np
import scipy.optimize as opt
from scipy import special
from tqdm.notebook import tqdm # Optional: for a progress bar

n_simulations = 1000
beta_hats = np.zeros((n_simulations, 2))
beta_stes = np.zeros((n_simulations, 2))

# True parameter values from the problem description
beta_0 = np.array([1, 1])

print(f"Running {n_simulations} Monte Carlo simulations with correct Logit␣
↪estimation...")

# Loop for the Monte Carlo simulation
for i in tqdm(range(n_simulations)):
    # 1. Draw a new sample
    y_draws, x_draws = LogitDraws()
    X = np.hstack((np.ones((x_draws.shape[0], 1)), x_draws))
    y = y_draws.flatten()

    # 2. Calculate MLE for beta using Logit likelihood
    initial_beta = np.zeros(X.shape[1])
    result = opt.minimize(
        neg_log_likelihood_logit, # Use the correct logit function
        initial_beta,
        args=(y, X),
        method='BFGS'
```

```
    )

    if not result.success:
        beta_hats[i, :] = np.nan
        beta_stes[i, :] = np.nan
        continue

    mle_beta = result.x
    beta_hats[i, :] = mle_beta

    # 3. Calculate Asymptotic Variance (Standard Errors) for Logit
    z = X @ mle_beta
    p = special.expit(z)
    epsilon = 1e-9
    p = np.clip(p, epsilon, 1 - epsilon)

    p_reshaped = p.reshape(-1, 1)
    y_reshaped = y.reshape(-1, 1)

    # Use the correct score for the Logit model: g = (y - p)X
    g = (y_reshaped - p_reshaped) * X
    J = g.T @ g

    try:
        asymptotic_var_cov = np.linalg.inv(J)
        std_errors = np.sqrt(np.diag(asymptotic_var_cov))
        beta_stes[i, :] = std_errors
    except np.linalg.LinAlgError:
        beta_stes[i, :] = np.nan

print("Simulations finished.")

# 4. Report how many times beta_0 was in the 95% CI
z_score = 1.96

# Calculate lower and upper bounds for both parameters across all simulations
lower_bounds = beta_hats - z_score * beta_stes
upper_bounds = beta_hats + z_score * beta_stes

# Check coverage for beta_1 (the intercept)
coverage_beta1 = np.sum((lower_bounds[:, 0] <= beta_0[0]) & (upper_bounds[:, 0]␣
↪>= beta_0[0]))

# Check coverage for beta_2 (the coefficient on X)
coverage_beta2 = np.sum((lower_bounds[:, 1] <= beta_0[1]) & (upper_bounds[:, 1]␣
↪>= beta_0[1]))
```

```python
print("\n--- Coverage Results (Correct Logit Model) ---")
print(f"Out of {n_simulations} iterations:")
print(f"The 95% CI for the intercept (beta_1) contained the true value of␣
↪{beta_0[0]} a total of {coverage_beta1} times.")
print(f"The 95% CI for the coefficient (beta_2) contained the true value of␣
↪{beta_0[1]} a total of {coverage_beta2} times.")
print(f"Expected coverage for a 95% CI is approximately {0.95 * n_simulations}.")
```

```
Running 1000 Monte Carlo simulations with correct Logit estimation...

  0%|          | 0/1000 [00:00<?, ?it/s]

Simulations finished.

--- Coverage Results (Correct Logit Model) ---
Out of 1000 iterations:
The 95% CI for the intercept (beta_1) contained the true value of 1 a total of
955 times.
The 95% CI for the coefficient (beta_2) contained the true value of 1 a total of
964 times.
Expected coverage for a 95% CI is approximately 950.0.
```